RobotStudio: Exercise 3
Using an arc welding gun

Applied Robotics & Applied Robotics for Architects
FRTN85/FRTN80
Faculty of Engineering, Lund University

September 2025

Contents

1 Introduction

In this exercise you will program the ABB IRB140 robot to perform motions required for welding plates (workpieces). These workpieces are arranged in a structure similar to those found in steel bridges or shipyards. Through this exercise, you will familiarize yourself with the following general skills:

- programming of a task which requires precise control of both target positions and paths between them
- performing collision checks between objects (primarily the robot and its environment)
- basic modeling of the simulated robot

The text of this exercise is the least detailed and verbose out of all of the exercises. There will be significant differences between your and other students' implementations.
You will nearly certainly have to modify and tune your initial implementation to reach a satisfactory solution (one that would work on the real robot). This will give you valuable experience for dealing with future robotic programming tasks.

2 Simulated workstation setup

Create new folders and a template with the *IRB140 6kg 0.81m* robot as before. Wait for *Controller status* in the lower right corner of the screen to turn green. Using *Import Library*, import the pre-defined objects used for the exercise. They are located in:

This PC\shared\Courses\control\FRTF20_Applied_Robotics\RobotStudio_exercises_2019\Lab3 Weld

The names of the objects to import are

- *rob-table*
- *welding-gun*
- *welding-case1*

Set the position of the robot on the table by putting its z coordinate to 670 mm.

Unlike before, you can not directly attach the end-effector (welding gun in this case) to the robot. This is because neither the welding gun model, nor the robot model, include the tool-changer which sits between the robot and an end-effector. This problem permits an easy solution — we just need to offset the welding gun from the flange by 40 mm (the flange is where the tool changer is attached).

Of course, you could simply attach the welding gun and offset it by 40 mm. However, in simulation this would result in a hole between the flange and the welding gun. To get a visual representation of the tool-changer, you will create a cylinder in RobotStudio and attach it to the flange. This way there is not going to be a hole between the flange and the welding gun after the offset.

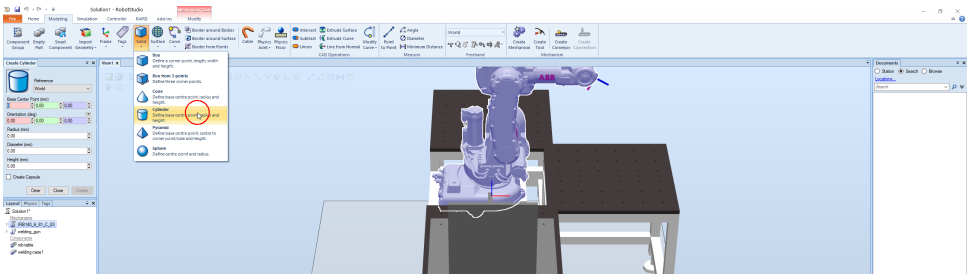


Figure 1: Location of the *Cylinder Solid*

To create the cylinder, click on the *Modelling* tab in the top menu and select *Solid* → *Cylinder*. Refer to Figure 1 to locate the button.

Put *Diameter* to 50 mm and *Height* to 40 mm. Create the cylinder and rename it to *tool_changer_40mm*. Furthermore, change its color by right-clicking on it and selecting *Modify* → *Set color* as shown in Figure 2. Choose an easily detectable color.

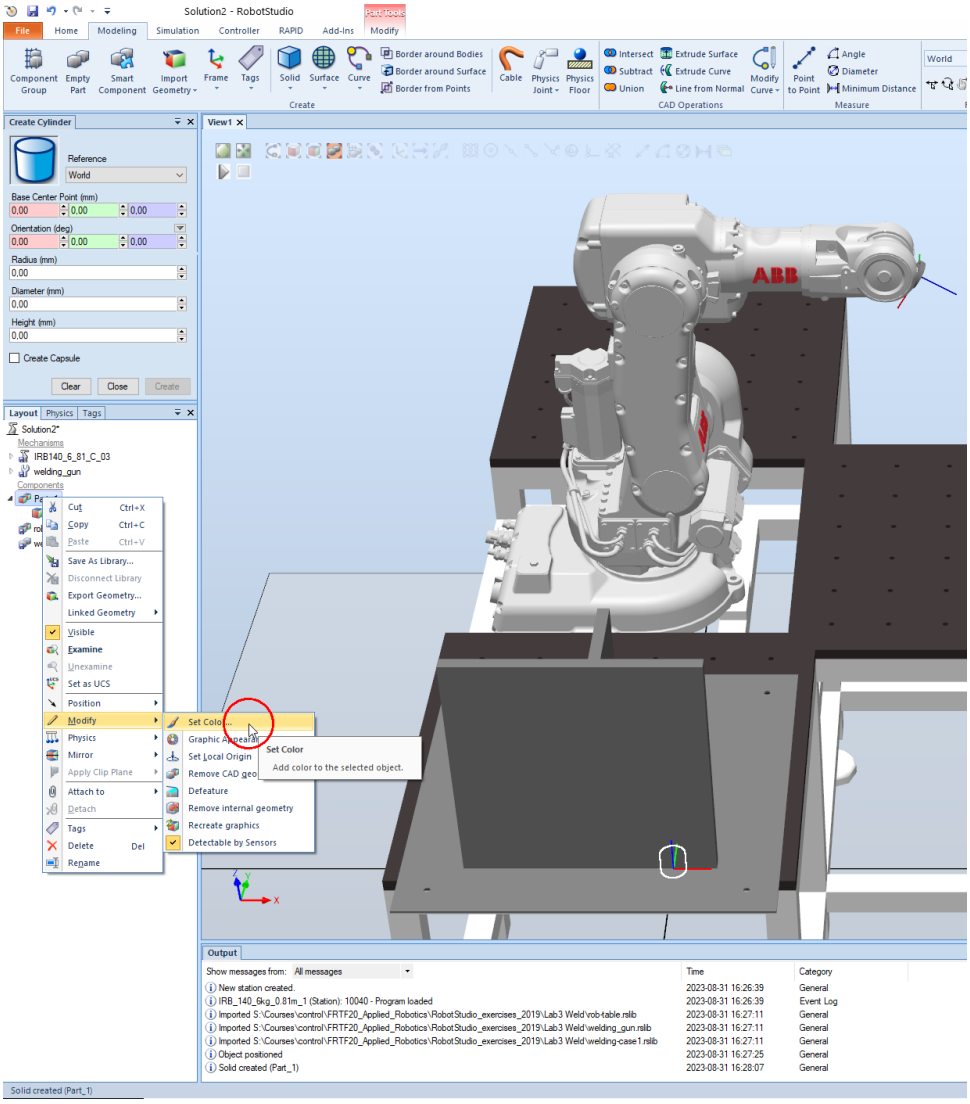


Figure 2: Locating the *Set color* option

Drag the cylinder to the robot in the *Layout* tab to attach it to the robot. After doing so, attach the welding gun to the robot in the same manner. Set the position of the

welding gun (by right-clicking on it in the *Layout* tab and selecting *Set position*) to be 40 mm away from the flange via the *Local* frame as *Reference*. The result should look like Figure 3. Notice the difference between the tip of the gun and the frame. This needs to be fixed.

Remark. In some setups, the reference frame is already correctly attached at the tip of the welding gun. If that is the case, you can skip the steps below and continue in Section 3.

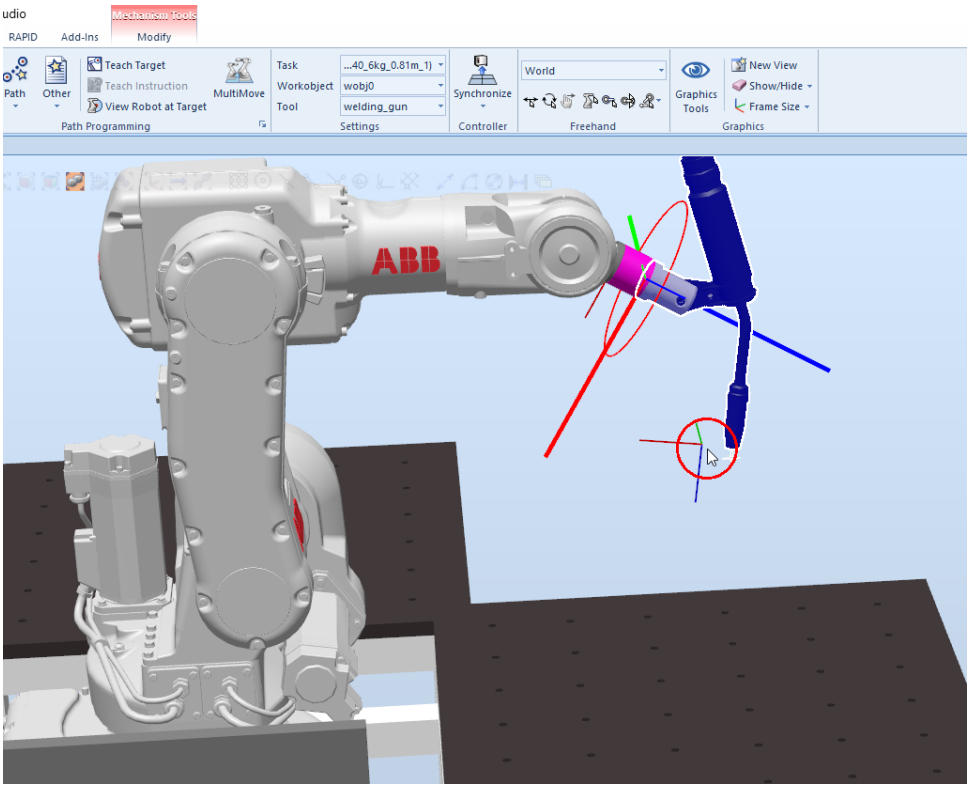


Figure 3: Situation after offsetting the welding gun away from the flange. Notice the difference between the tip of the gun and the frame. This needs to be fixed.

Right-click on the welding gun in the *Paths & Targets* tab and select *Set position* as indicated in Figure 4.

Select the *Snap Center* option in the toolbar on top of the simulation screen as indicated in Figure 5.

Next, left-click on one of the coordinate fields. Zoom in and click on the tip of the welding gun. The frame should now look like the one in Figure 6.

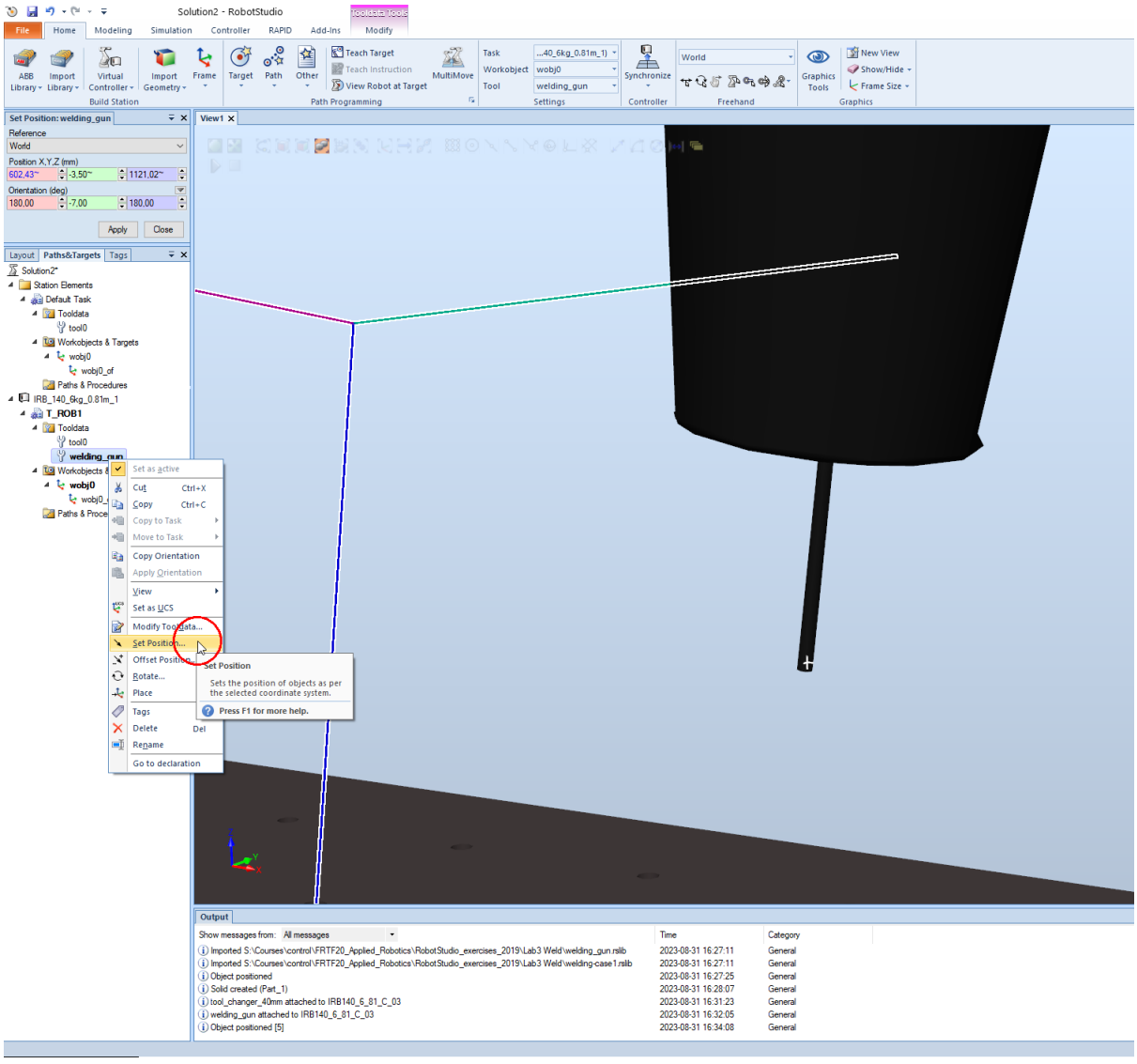


Figure 4: Setting the position of the welding gun tip reference frame

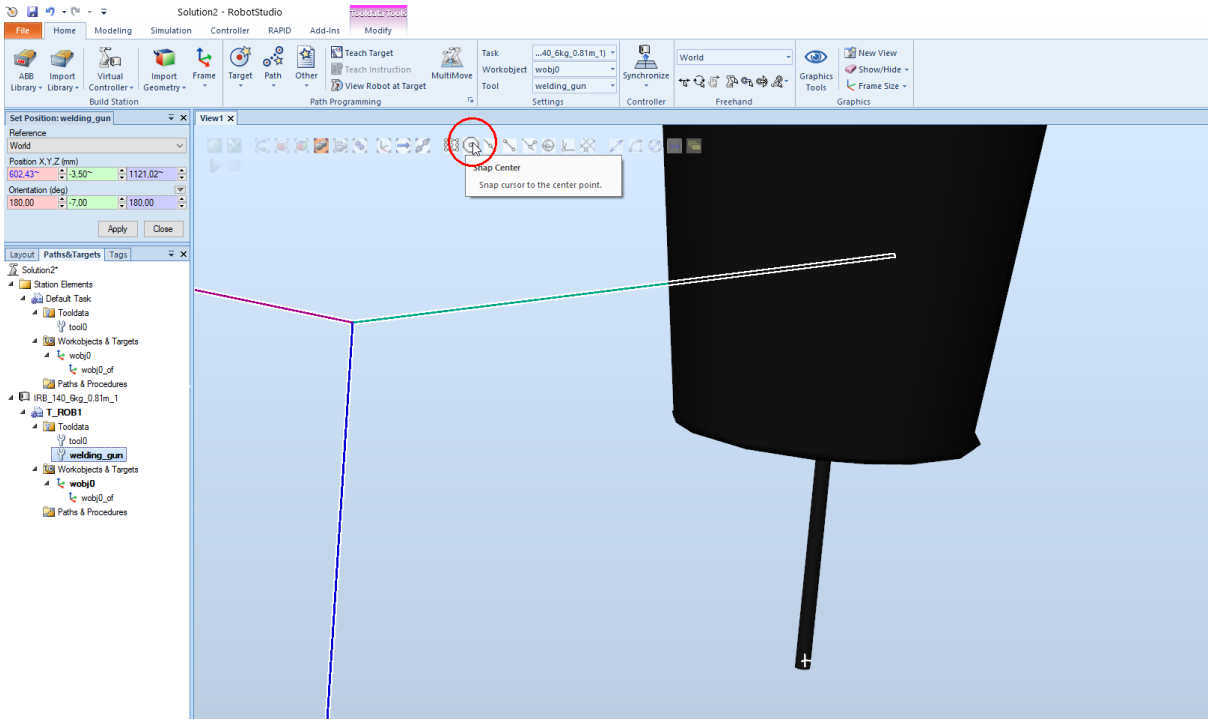


Figure 5: Location of the *Snap Center* tool

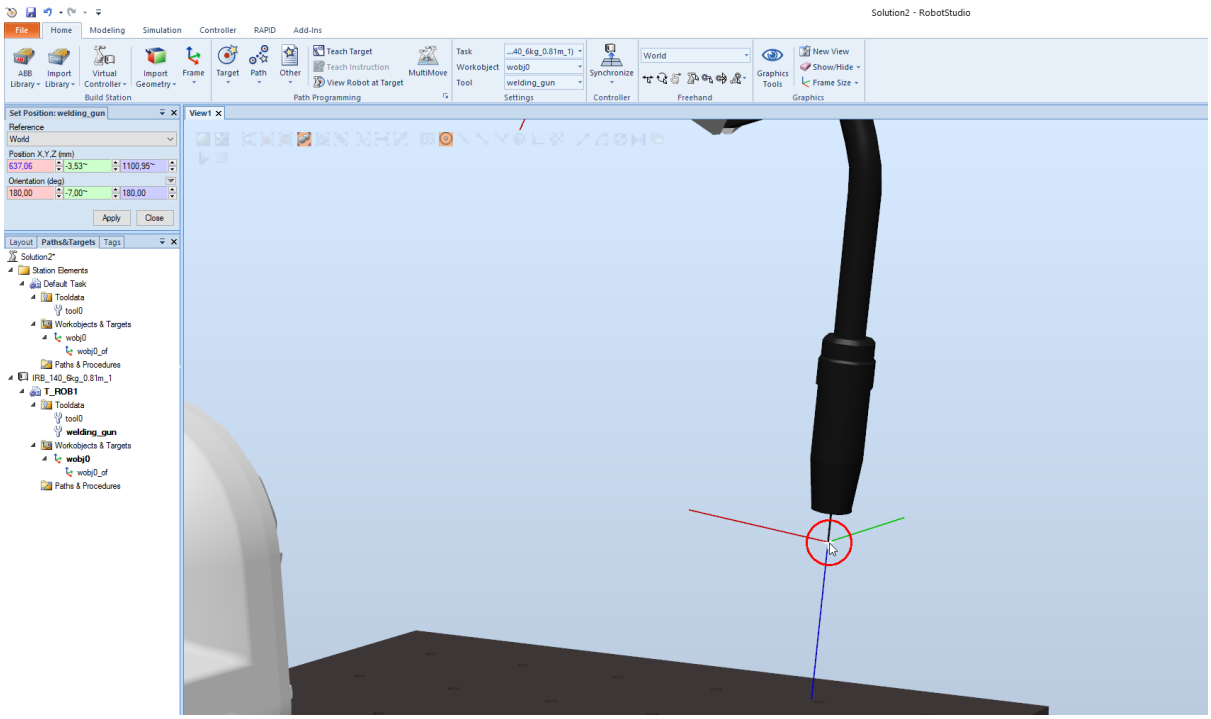


Figure 6: A corrected frame at the tip of the welding gun

3 Creating targets and paths for the welds

3.1 Creating the workobject

Create a workobject for the part to be welded. Set the *User Frame* values to $(300, -400, 124)$ and rotate it by $(-180, 0, -90)$ degrees. Keep the *Object Frame* values at 0. The result should look like Figure 7.

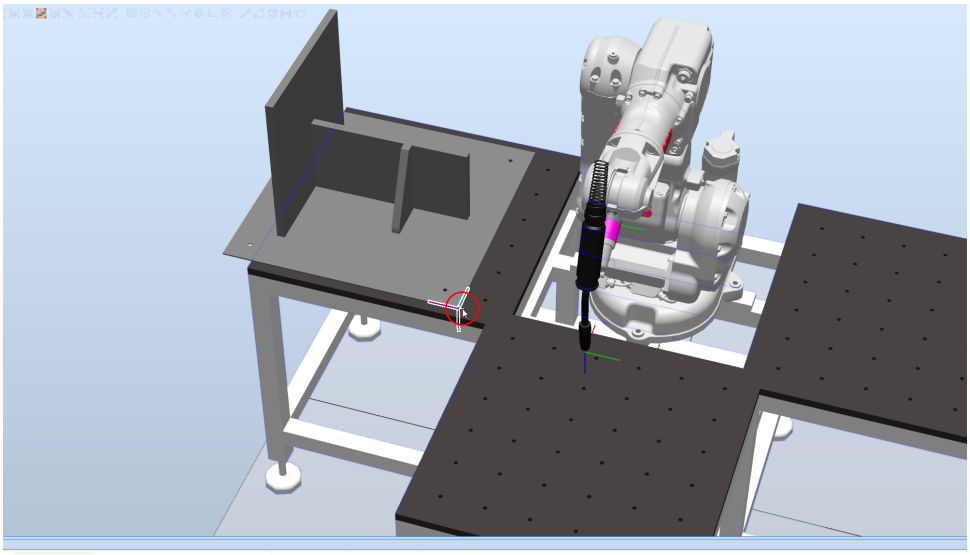
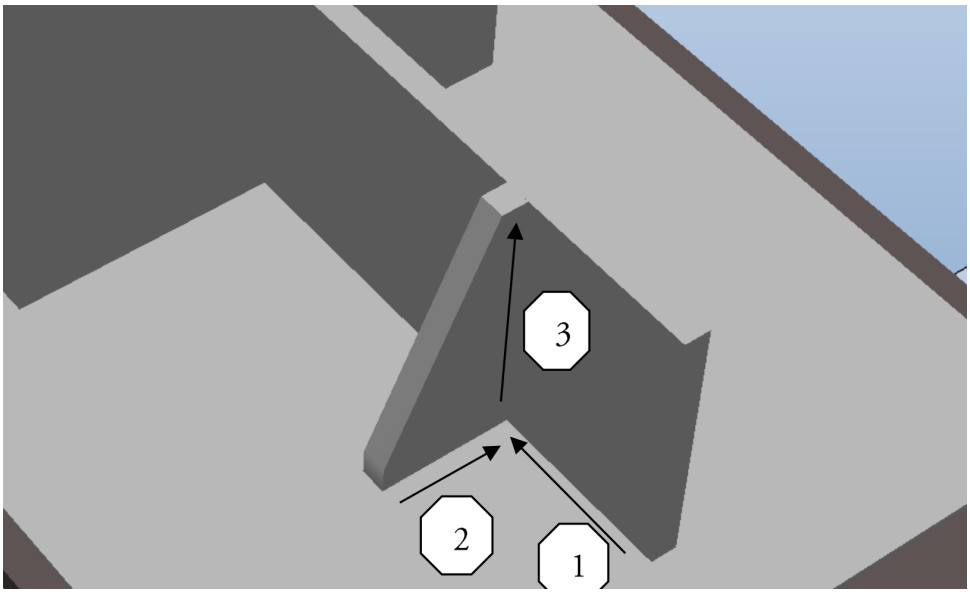


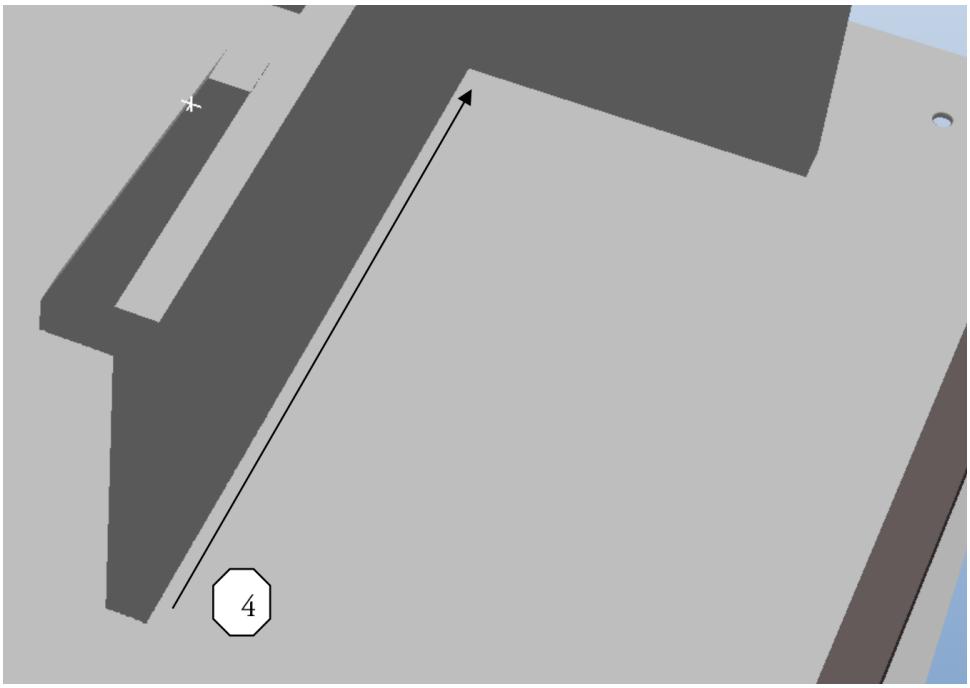
Figure 7: Desired location for the welding part workobject

3.2 Creating paths for the welds

The welds are 4 straight-line paths defined in Figure 8a and Figure 8b. Your task is to create all these paths (and transitions between them) according to specifications given below.



(a) First 3 welds



(b) The 4th weld on the other side of the part

Figure 8: Welds are the 4 straight-line paths depicted in these images

To correctly perform a weld, the welding gun has to maintain a distance of about 30 mm above the part to be welded. Thus a fine zone should be used. Since all our welds are straight lines, you should use *MoveL* along them. The orientation of the welding gun should be 45 degrees to the plates around the weld (since the plates are perpendicular to each other). The $v50$ velocity should be used for the weld movements

to emulate the speed requirements in real welding scenarios (more on that in Section 5). Overall, the general workflow for defining a weld path goes as follows:

General workflow for defining a weld path

1. define the starting target of the weld in question:
   (a) get the robot near the target via *Mechanism Linear Jog* and/or *Mechanism Joint Jog* and teach this target via *Teach Target*
   (b) select the welding gun as the jogging reference (refer to Figure 9) to achieve:
      - the orientation of the welding gun of 45 degrees between the plates
      - the tip of the welding gun being 30 mm away from the weld line
      - **tip1:** use an appropriate fixed step size when jogging
      - **tip2:** get the point of the welding right to just touch the welding (where the plates meet), and then move it back by 30 mm in the welding gun reference; refer to Figure 11
   (c) teach this target as the starting target of the weld
2. repeat the steps in 1 to teach the target at the end of weld and one close to it
3. define a *MoveL* which keeps the distance and orientation of the welding gun to the weld
4. set the velocity and the zone appropriately
5. use *Auto Configuration* and check that there are **no singularities, hitting of joint limits, or collisions** (consult Section 4 to see how to do collision checking) along the path, and if there are collisions:
   - try adding intermediate points to the weld
   - start the robot from different poses at any or all of the targets along the path by jogging the robot and (re)creating targets
6. once the path works, save the station

Before starting with the first path, consult Section 3.3 for some tips and tricks to help you in the process. Start by creating the *home* position.

Create the paths and synchronize to RAPID by adding one path at a time. Make sure that the first target is a *MoveJ*, otherwise a synchronization error is likely to occur when trying to *Synchronize to Station*.

Note: If stuck in the sync process make a *P-start*, delete *Module1* containing your program, create a *New Module* and name it *Module 1*. This reset should fix the issue. If it does not, change all move instructions to *MoveJ*. Include only *Save Paths* and start from there. Finally, add the *home* position to the last weld path at the end and make it a *MoveJ* instruction. When the *Synchronization to RAPID* is working, add a *main* procedure (either in a separate module or as a procedure in *Module1*) and run the simulation. You might notice some strange motions and suspected collisions between

weld paths. You will need to do some fine-tuning to check and manage collisions, which will be discussed in Section 4.

Once you have created all of the paths, **check that there are no collisions or singularities along across the entire program**. This will likely happen if you have not defined additional targets between different paths, so add those to fix the issues. Read Section 4 to learn how to do collision detection in RobotStudio.

3.3 Some tips and tricks

You will be doing quite a bit of jogging to get usable targets. To help the process try using other jogging methods available in RobotStudio. The one already mentioned is jogging orientation while having the welding gun as reference. Selecting this option is displayed in Figure 9. Other options include dragging arrows for translation or circles

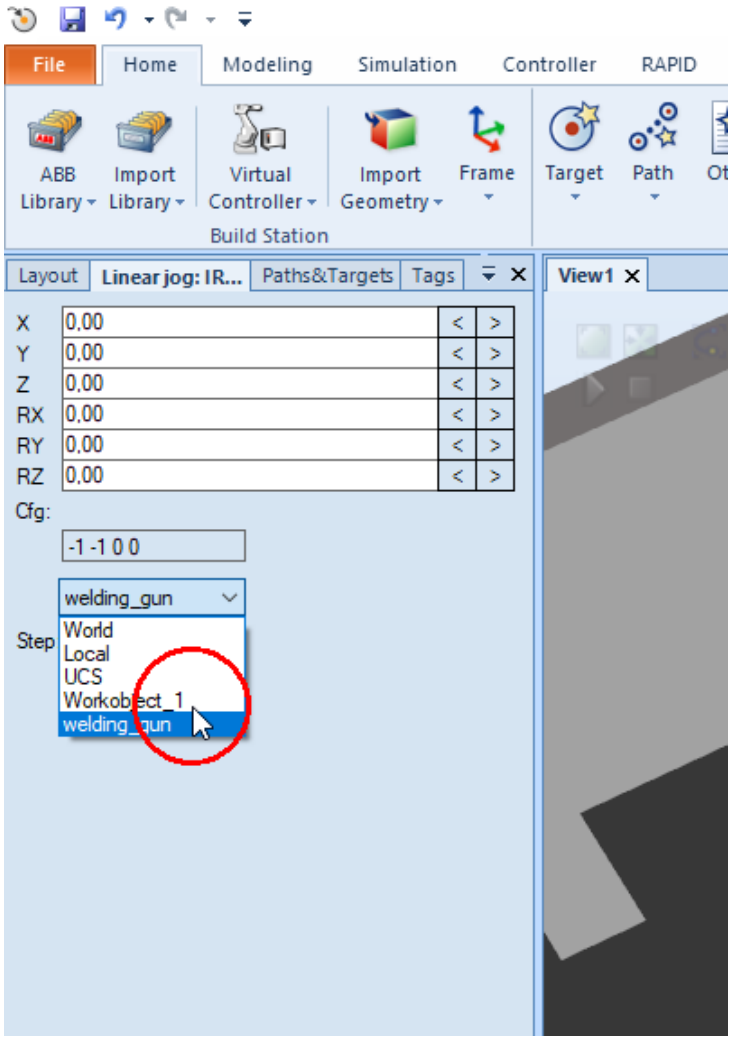


Figure 9: Selecting welding gun as reference for jogging

for orientation using your mouse. This is called *Freehand Jog* in RobotStudio. Selecting these options is shown in Figure 10. While trying to find get the tip of the welding gun 30 mm away from where the plates meet, you can instead make the tip of the welding

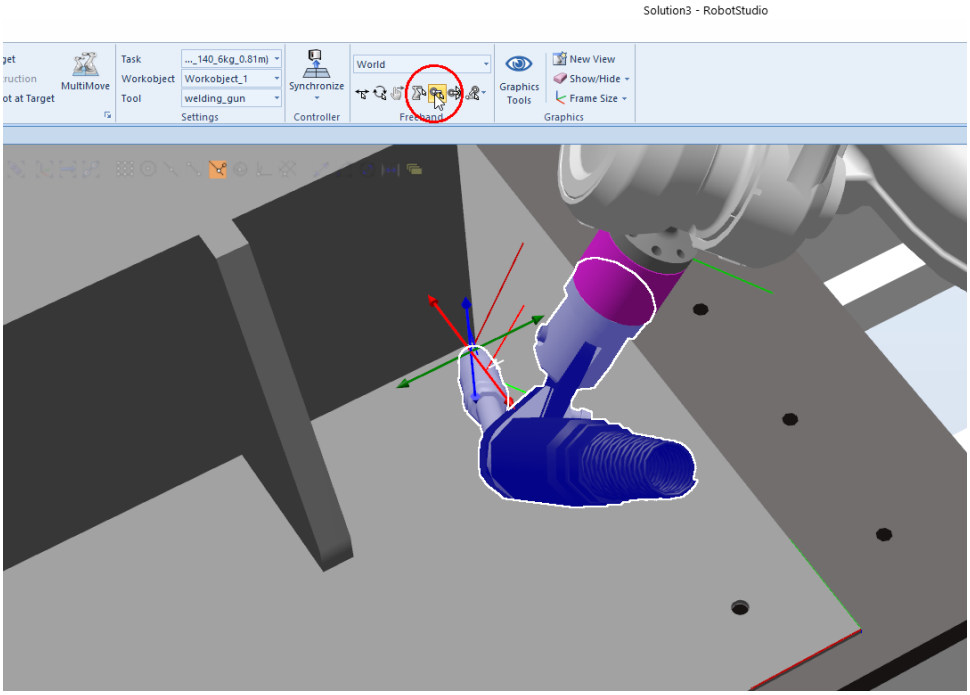


Figure 10: You can try out some of the *Freehand Jog* options available. They will allow you to jog by dragging arrows and circles which appear around objects.

gun just touch the point, and then translate it back 30 mm in the z coordinate of the welding gun. This is shown in Figure 11.

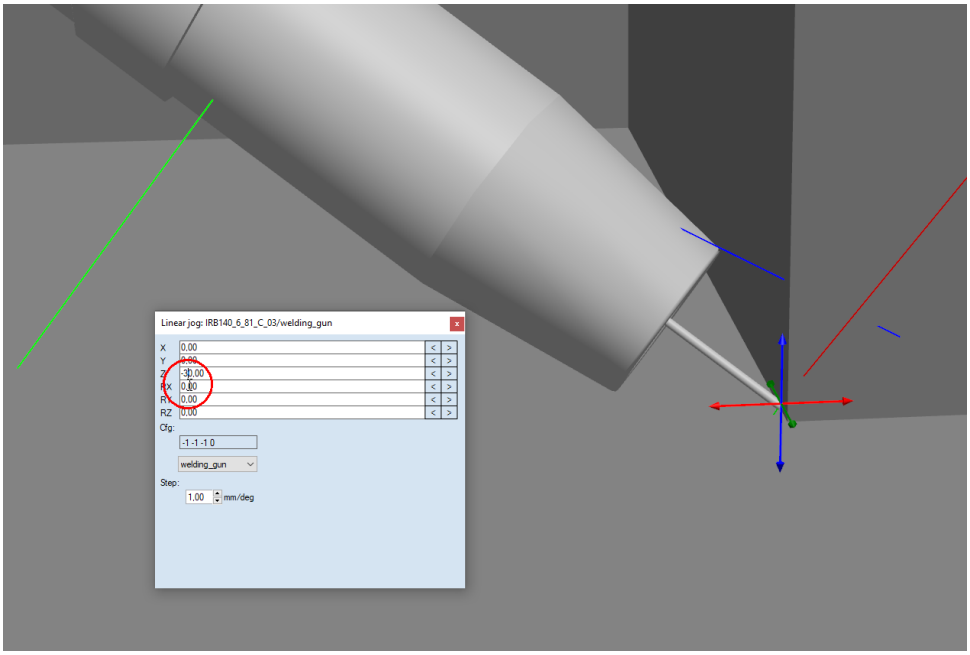


Figure 11: Getting the distances right by going to a clearly visible point and then translating back via linear jogging

4 Collision checking

The simulation environment helps us detect collisions and other problems by allowing us to visually inspect paths. Additionally, RobotStudio (and many other simulators) can perform collision checking between selected objects. This is helpful and important because some collisions are really difficult to spot, and because even those obvious on close inspection can easily be missed while looking at the simulation.

In this exercise **you will use two collision sets**:

1. one to check for self-collisions, i.e., *between the robot and the welding gun*
2. one to check for collisions between the robot and welding gun and the weld object

A collision set is created as follows: In the *Simulation* tab, select *Create Collision Set*. To locate the button, refer to Figure 12.

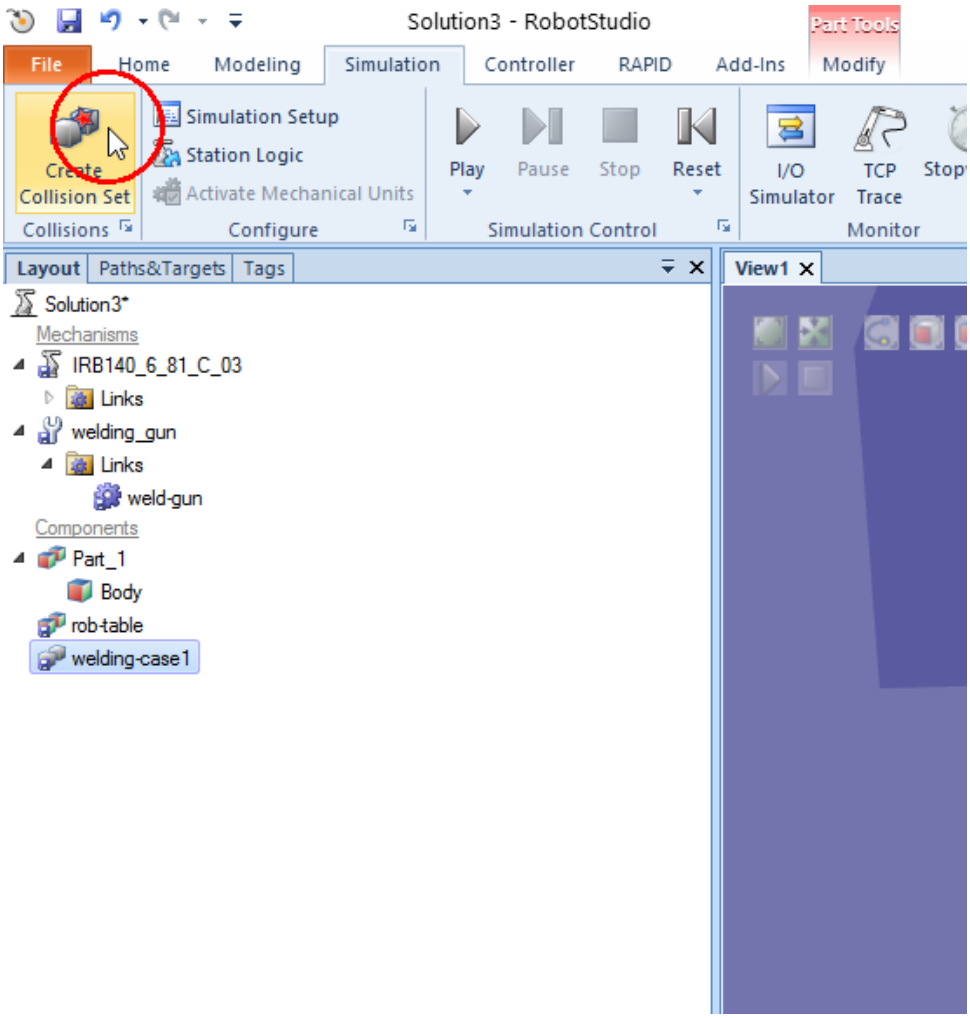


Figure 12: Location of the *Create Collision Set* button

After clicking the button, *CollisionSet_1* will appear in the *Layout* tree. First create the self-collision checking set. Expand the collision set and drag the *Links Base, Link1, . . . , Link5* to *ObjectsA* and the welding gun to *ObjectsB*.

When you detect a collision, you will want to change targets. You can do this as follows. Move to the target you want to change for example by selecting *Jump to Target*. Jog the robot to a new location. Teach a new target. Right-click on the target you want to change and select *Modify Target* → *Place* → *Frame*. In the box *Select Frame* click on the new modified target you just created, and click on *Apply*. You may then delete the last made target.

Note the following

- *Link6* is not included because that would create a collision all the time as that link is connected to the welding gun.
- It does not matter what objects are dragged to *ObjectsA* or *ObjectsB*.
- Collision sets are computer demanding. You may notice that the simulation runs slower. Hence, you probably do not want to have the collision checking on at all times. Collision sets can be made active or inactive by right-click on the respective set.

Now create the robot with the welding gun and welding object collision checking set. Do this by creating one more collision set, and drag both the robot and the welding gun to *ObjectsA*, and the welding case to *ObjectsB*. Run the simulation again and you will most likely notice a number of collisions. Collisions will be highlighted in red and recorded in the *Output log*. Go through the collisions one by one and fix the problems as indicated above.

5 Typical welding scenarios in practice and reflection

End-effector speed in real welding Typical welding speed for a case like this is usually in the order of 4-8 mm/s, in several layers and performed in a weaving pattern. Thinner plates are produced with a higher welding speed, in the order of up to 25 mm/s, but this is highly dependent on many interrelated parameters including plate material, thickness, joint design, tolerances and so on.

Collision checking in practical applications In this exercise the welding station is comparably simple. In a realistic scenario, great care must be taken when working with collision sets. The number of collision sets and collision checking overall needs to be limited as it is not possible to perform collision checks on all possible pairs of objects due to the computational constraints.

Difficulties in path design Creating the targets and the paths was probably a bit tricky in this exercise due to the given object to be welded. This is also typical in real situations. However, in this case, you were dealing with only a few targets compared to the number encountered in practice where they can be in the hundreds. A real workpiece may include between 20-100 welds, most of which are at difficult positions to get to. This is because many targets or weld paths generate configurations close to joint limits, collisions or singularities. Such situations are especially common for

processes like arc welding that where the robot tool needs to go through precisely defined paths.

Overall, real welding includes a lot of process-related considerations which put great demand on the programmer.

Think about the following:

- What amount of time is required to make a full arc welding program (or for any similar work process)?
- How can you verify that the tool center point of the real welding gun is as defined in the program without doing a full tool calibration?
- Read the procedure for defining workobject frames described below and make sure you understand the mathematics behind it.

5.1 Defining and calibrating workobjects

In practical situations, it is convenient to add some simple routines and tests to assure that workobjects and tool definitions seem to be correct, e.g., making a reorientation around the tooltip and verify that it stays at same position; the easiest way to see if close to stationary reference. The process of finding out and tuning the correct values of parameters of physical objects is called calibration.

The following description is taken from the ABB FlexPendant (tool used to control the real robot) operational manual and is given in Figure 13 and Figure 14.

Defining the work object coordinate system <-

-

Overview

Defining a work object means that the robot is used to point out the location of it. This is done by defining three positions, two on the x-axis and one on the y-axis.

When defining a work object you can use either the user frame or the object frame or both. The user select frame and the object frame usually coincides. If not, the object frame is displaced from the user frame.

How to select method

This procedure describes how to select method for defining either user frame or object frame or both. Note that this only works for a user created work object, not the default work object, wobj0. Defining work object can also be done from the **Program Data** window.

	Action
1	On the **ABB** menu, tap **Jogging**
2	Tap **Work object** to display the list of available work objects.
3	Tap the work object you want to define, then tap **Edit**.
4	In the menu, tap **Define.....**
5	Select method from the **User method** and/or the **Object method** menu. See How to define the user frame and How to define the object frame

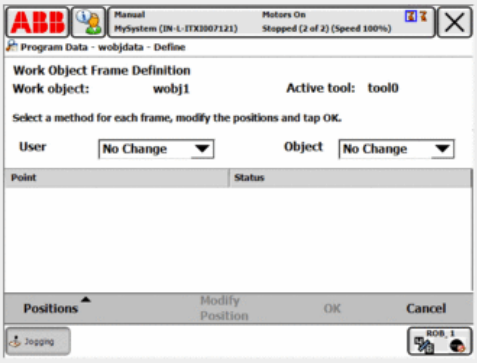
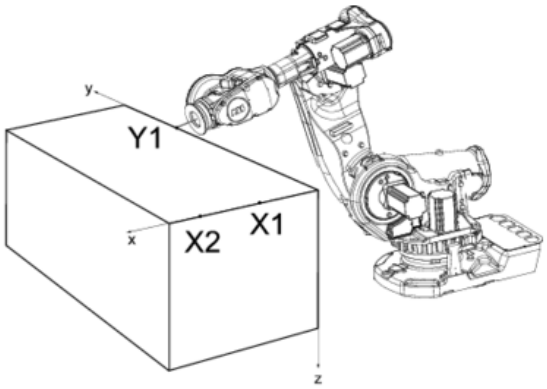


Figure 13: Manual page 1

How to define the user frame

This section details how to define the user frame.

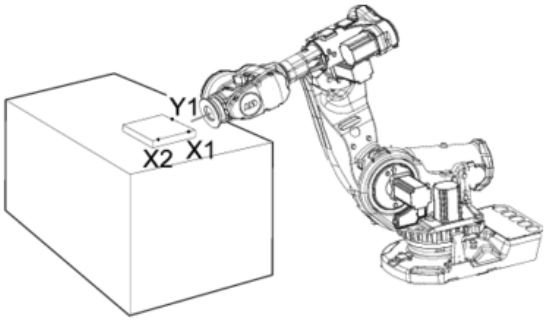


The x axis will go through points X1-X2, and the y axis through Y1.

	Action	Info
1	In the **User method** pop up menu, tap **3 points**.	
2	Press the enabling device and jog the robot to the first (X1, X2 or Y1) point you want to define.	Large distance between X1 and X2 is preferable for a more precise definition.
3	Select the point in the list.	
4	Tap **Modify Position** to define the point.	
5	Repeat steps 2 to 4 for the remaining points.	

How to define the object frame

This section describes how to define the object frame if you want to displace it from the user frame.



The x axis will go through points X1-X2, and the y axis through Y1.

	Action
1	In the **Object method** pop up menu, tap **3 points**.
2	See steps 2 to 4 in the description of How to define the user frame.

Figure 14: Manual page 2